Comprehensive Viva Voce & Evaluator Examination Guide

Embedded Electronics Fundamentals for Computer Science Engineers

This guide prepares Computer Science students to confidently answer technical questions posed by Electronics & Communication (EC) evaluators regarding the **Smart Pill Box** embedded project.

Part 1: Electronics & Circuit Physics (The "EC Trap" Questions)

Q1: Why are pull-up resistors required on the I2C bus, and how do you calculate their value?

Answer:

*I2C pins operate in **Open-Drain (`AF_OD`)** mode. Transistors on the chip can only pull the signal line actively to Ground (Logic 0). They cannot drive the line HIGH (Logic 1). Pull-up resistors connected to V_{DD} passively restore the bus to HIGH when all devices release the bus.*

Calculation Formula:

The value of the pull-up resistor R_p is bounded by two physical constraints:

*1. **Maximum Value ($R_{p,\max}$)**: Limited by the RC rise time t_r and total bus capacitance C_b (traces + pin capacitances, typically $30-100\,\text{pF}$):*

$$R_{p,\max} = \frac{t_{r,\max}}{0.8473 \times C_b}$$

For $100\,\text{kHz}$ standard I2C, $t_{r,\max} = 1000\,\text{ns}$. If $C_b \approx 50\,\text{pF}$:

$$R_{p,\max} \approx \frac{1000 \times 10^{-9}}{0.8473 \times 50 \times 10^{-12}} \approx 23.6\,\text{k}\Omega$$

*2. **Minimum Value ($R_{p,\min}$)**: Limited by the maximum sink current ($3\,\text{mA}$) the NMOS transistor can sink while maintaining $V_{OL} \le 0.4\,\text{V}$:*

$$R_{p,\min} = \frac{V_{DD} - V_{OL}}{I_{OL}} = \frac{3.3\,\text{V} - 0.4\,\text{V}}{3\,\text{mA}} \approx 966\,\Omega$$

*Therefore, standard values between **$2.2\,\text{k}\Omega$ and $4.7\,\text{k}\Omega$** are optimal.*

Q2: What is the physical difference between a 5V-Tolerant (FT) pin and a Non-FT pin on the STM32?

Answer:

* ***Non-FT Pins (`PA0` to `PA7`, `PB0`, `PB1`)**: Connect to internal analog multiplexers (ADC/DAC). They have two internal silicon electrostatic discharge (ESD) clamping diodes: one to GND and one to V_{DD} (3.3V). If you apply 5V, the upper diode conducts forward current directly into the 3.3V supply rail, causing substrate latch-up and burning out the analog inputs.*

* ***FT Pins (`PB6`, `PB7`, `PB12`, `PB14`, `PB15`)**: Have **no upper clamping diode to V_{DD}**. Instead, they feature an isolated parasitic N-well with a reverse breakdown threshold exceeding 6.0V, allowing them to connect safely to 5V open-drain pull-ups.*

Q3: Why does the DS1307 RTC fail if powered by 3.3V instead of 5V?

***Answer**:*

*The DS1307 has an internal **analog power-fail comparator circuit**. Its reference threshold is:*

$$V_{PF} = 1.25 \times V_{BAT} = 1.25 \times 3.0\,\text{V} = 3.75\,\text{V}$$

If V_{CC} drops below V_{PF} (e.g. connecting to the $3.3\,\text{V}$ rail):

1. The comparator trips to protect non-volatile memory.

2. The internal logic disconnects V_{CC} and switches to the coin battery (V_{BAT}).

*3. It **locks out the I2C interface**, holding SDA and SCL in high-impedance and refusing to send ACK bits.*

*Therefore, V_{CC} must be connected to the **5V pin** (fed by USB). Because the STM32 I2C pins **PB6** and **PB7** are FT-rated and pulled to 3.3V, communication is completely safe.*

Q4: Why is a 32.768 kHz crystal used for the RTC instead of a common 8 MHz crystal?

***Answer**:*

*1. **Binary Division**: $32,768$ is an exact power of two (2^{15}). A chain of 15 toggle flip-flops divides this down to an exact $1.000\,\text{Hz}$ (1-second tick) using simple logic gates without rounding errors:*

$$\frac{32,768\,\text{Hz}}{2^{15}} = 1.000\,\text{Hz}$$

*2. **Power Conservation**: Dynamic CMOS power consumption is proportional to frequency ($P = C \cdot V^2 \cdot f$). Running the oscillator at $32.768\,\text{kHz}$ draws less than $500\,\text{nA}$, allowing a small $220\,\text{mAh}$ CR2032 coin cell to sustain the clock for over **10 years**.*

Q5: How did you implement switch debouncing without hardware RC filters?

***Answer**:*

Pushbutton metal contacts bounce mechanically for $5\text{ to }20\,\text{ms}$ upon closing. While an RC low-pass filter ($R = 10\,\text{k}\Omega, C = 100\,\text{nF}, \tau = 1\,\text{ms}$) can filter this in hardware, it adds passive component cost.

*We implemented **non-blocking software edge-detection debouncing** using **`HAL_GetTick()`**:*

```c
if (lastButtonState == GPIO_PIN_SET && currentPinState == GPIO_PIN_RESET) {
if (now - lastPressTimestamp > 200) { // 200 ms lockout threshold
lastPressTimestamp = now;
// Register valid user click
}
}
```

*This accepts the instantaneous first contact edge and locks out subsequent bounce transitions without using a blocking **`HAL_Delay()`**.*

Q6: How does the OLED display generate 7.5V to 9V from a 3.3V power supply?

> ***Answer**:*
>
> *Organic LEDs (OLEDs) require forward voltages around $7-9\text{V}$ to overcome their bandgap and emit photons. The SSD1306 controller incorporates an internal **switched-capacitor DC-DC charge pump**:*
>
> ** During phase 1, external flying capacitors across pins `C1P/C1N` and `C2P/C2N` charge in parallel from the 3.3V supply.*
>
> ** During phase 2, internal MOS switches stack the charged capacitors in series, stepping up the potential to $\approx 7.5\text{V}$.*
>
> ** In firmware, this is commanded during `ssd1306_Init()` via opcodes `0x8D, 0x14`.*

Part 2: Protocols, Microcontroller Architecture & Firmware

Q7: How can both the OLED and RTC share the exact same pins (PB6 and PB7) without bus collisions?

> ***Answer**:*
>
> *I2C is a synchronous, multi-drop, address-delimited serial bus. Every bus transfer begins with a START condition followed by a **7-bit device address**:*
>
> ** **SSD1306 OLED**: Address `0x3C` (`0x78` shifted for write).*
>
> ** **DS1307 RTC**: Address `0x68` (`0xD0` shifted for write).*
>
> *Only the peripheral whose internal hardware address matches the broadcast address responds with an **ACK (Acknowledge)** bit by pulling SDA LOW on the 9th clock pulse. All other slaves ignore subsequent data bytes until a STOP condition is asserted.*

Q8: What are the differences between SPI and I2C in this project?

> ***Answer**:*

Parameter	I2C1 (OLED & DS1307)	SPI1 (MicroSD Card)
Physical Lines	2 wires (`SCL`, `SDA`)	4 wires (`SCK`, `MISO`, `MOSI`, `CS`)
Addressing	Software 7-bit addressing	Hardware Chip Select pin (`PA4`)
Bus Topology	Open-drain with external pull-ups	Push-pull (Active HIGH/LOW)
Clock Speed	Standard 100 kHz	9 MHz data / 281 kHz init
Duplex	Half-duplex (shared data line)	Full-duplex (separate MISO & MOSI)
Throughput	~10 KB/s	~1.1 MB/s

Q9: Why is Flash Latency set to 2 wait states in SystemClock_Config()?

> ***Answer**:*
>
> *The internal Flash memory of the STM32F103 cannot deliver instructions faster than $24\,\text{MHz}$ without read errors. Since our CPU core ($SYSCLK$) runs at $72\,\text{MHz}$, the Flash access controller must insert wait states (clock delay cycles):*

** $0-24\text{ MHz}$: 0 Wait States*

** $24-48\text{ MHz}$: 1 Wait State*

** $48-72\text{ MHz}$: **2 Wait States (`FLASH_LATENCY_2`)***

If set to 0 wait states at 72 MHz, the instruction prefetcher reads garbage opcodes and immediately crashes into a `HardFault`.

Q10: How does the pure-C SSD1306 driver avoid third-party library overhead?

***Answer**:*

*Libraries like Adafruit GFX or U8g2 consume significant Flash memory and pull in large C++ font hierarchies. Our pure-C driver maintains a local **1,024-byte SRAM monochrome framebuffer** (`128 * 64 / 8 bytes`), consuming only **5% of the 20 KB SRAM**. Drawing functions compute byte and bit offsets using fast bit-shift math:*

$$\text{Byte Index} = x + \left(\frac{y}{8} \times 128\right),\quad \text{Bit Mask} = 1 \ll (y \pmod 8)$$

The full display buffer is transferred to the physical GDDRAM across 8 pages in burst mode.

Part 3: Safety Logic, MicroSD Storage & Dosage History

Q11: How does the system prevent accidental medication errors?

***Answer**:*

*The system implements a **two-stage interlocked verification state machine**:*

1. When an alarm triggers (e.g. Pill 1 at 07:30), the screen displays `"TAKE: >> PILL 1 <<"`, and LED 1 flashes with a 500 ms alert tone.

*2. If the patient presses Button 2 (Pill 2) instead of Button 1, the firmware detects the mismatch, enters a high-urgency **150 ms fast alert**, flashes an inverted video `"WRONG PILL!"` warning, and logs `SD_EVENT_WRONG_BUTTON`.*

3. The alarm only silences and confirms when the patient presses the correct button for that specific pill.

Q12: How does the On-OLED 3-Day Dosage Viewer operate?

***Answer**:*

Selecting `> 4. VIEW LOGS` in the Settings menu opens a 1-page-per-day history viewer:

** **Page 1 (Today)**: Shows today's date, whether Pill 1 and Pill 2 are `TAKEN` (with recorded time, e.g. 07:31), `PENDING` (time has not arrived), or `MISSED`.*

** **Page 2 (Yesterday, -1d)** & **Page 3 (Day Before, -2d)**: Displays compliance history for the preceding 48 hours.*

** **Navigation**: Button 1 (`PB12`) cycles through the 3 pages, and Button 3 (`PB15`) returns to the menu. Dates are calculated using an embedded calendar algorithm that accounts for month boundaries and leap years.*

Q13: How does persistent alarm configuration on the MicroSD card work?

***Answer**:*

*When the user edits and saves alarm schedules in the menu, the firmware writes a 512-byte block to **Sector 1000** of the SD card containing:*

```text
PILLBOX_CFG_V1 P1=07:30 P2=19:00\r\n
```

At power-on, `SD_LoadConfig()` reads Sector 1000 using `CMD17`. If the `PILLBOX_CFG_V1` header is present, it parses the hours and minutes, restoring user schedules across power outages without needing an external computer.